Slide 1: Load Balancing is the decision to which node of a network of nodes that work is done as well as what tasks need to be performed. It also refers to the transfer of computing tasks from nodes that are more heavily loaded to nodes that are less loaded. Some tradeoffs to be considered are the uniform distribution of work as well as the network that the nodes exist on and its capabilities.

Slide 2: There are three main challenges to load balancing. The structure of the distributed application isn't uniform and different processes require different computing capacities. An example is in a microservices architecture certain tasks of a service may require more or less computing resources. Sending every task to a node of the same computing capacity doesn't take into account the challenge of the task. The second challenge is the cluster isn't uniform and different computing nodes have different performance. To ensure that all nodes in the computing cluster are used to the best of their capacity routing the right tasks to the appropriate nodes is challenging. It requires knowledge of the amount of capacity a problem needs in relation to other tasks that may need to be balanced. The third challenge I am less familiar with but it is that internode communication is non-uniform and different nodes have different bandwidths. I think this challenge is similar to something Izzy mentioned during out last presentation about how in a distributed system the most likely point of failure and hardest thing to plan for is the network. There are three goals to load balancing. Cost Effectiveness, Scalability, and Priority. Cost effectiveness is providing a better system performance at a lower cost. Scalability and flexibility means the system that does the load balancing can be changed in size after some amount of time. Prioritization is the prioritization of resources or jobs based on the importance of the job.

Slide three: There are two primary classes of load balancing depending on system state. With static algorithms the current state of the node is not taken into account, a prior knowledge base is required about each node such as statistics, and user requirements. It is not flexible, scalable, or compatible with change. This is useful in predefined systems that change does not occur very often. The second class of load balancing algorithms are dynamic algorithms. These algorithms operate according to changes in node states. They collect, store and analyze information about system state. If a node fails it wont stop the rest of the network from working but can influence system performance. There are three classes of algorithms based on process initiation. Initiated by sender is an algorithm where the number of nodes is large and a node sends for initiation of the execution of the load balancing algorithms. Initiated by receiver is when the load balancing algorithm is identified by a receiver or server in the system and then the server initiates the load balancing algorithm. Symmetric is a mix of these two algorithms. Dynamic load balancing is divided into two types: centralized and distributed. A centralized system is when one node or group of nodes are responsible for the management and distribution of information. This kind of system is not fault tolerant, the load balancing system itself can be overloaded, and it is useful in small networks. I struggle a little more with distributed systems but each node builds a load vector independently all nodes are responsible for the load distribution and content of their own local db. There are large communication costs and a large complexity cost. Distributed is further broken up into cooperative, each node operate for a common goal, and non-cooperative each node operates independently for local goals

Slide 4: Data collection in a load balancing system is divided into three categories. Data collection on need are algorithms that are used to collect information about capacity and when a node needs to be balanced. Periodic collection is used in centralized systems and distributed systems to initiate balancing. Change of state algorithms allow distributed algorithms to spread information about changing capacity.